Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial 2026/2027



Disusun oleh:

NopalLanguageProcessing

13524003	Faiq Azzam Nafidz
13524013	Anindya Naufal Pinasthika
13524139	Azri Arzaq Pohan

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **3**
Pembahasan **4**
A. State & Objective function 4
B. Pembangkitan Neighbor 7
C. Implementasi Algoritma 8
a. Hill-Climbing 8
I. Steepest Ascent Hill-Climbing 8
II. Hill-Climbing with Sideways Move 9
III. Random-Restart Hill-Climbing 10
b. Simulated Annealing 11
c. Genetic Algorithm 13
D. Visualisasi 13
E. Hasil Testing 14
F. Implementasi Bonus 15
a. Varian Alternatif Objective Function 15
b. Implementasi Seluruh Algoritma Hill-climbing 16
Kesimpulan **17**
Referensi **18**
Lampiran **19**
A. Tabel Pembagian Tugas 19

Deskripsi Persoalan

Tugas Besar 1 IF3170 Inteligensi Artifisial merupakan tugas besar yang membahas penyusunan konfigurasi paket pada sebuah truk untuk menyelesaikan permasalah *bin packing problem*, yakni bagaimana menyusun paket-paket ke dalam truk secara optimal menggunakan algoritma *local search* yang telah dipelajari dengan cara program menerima data truk dan paket lalu mencari susunan agar dapat menghasilkan nilai paket yang maksimal dimuat.

Setiap paket sendiri mempunyai dimensi, nilai, berat, status mudah pecah, ETA, dan orientasi untuk menjadi parameter yang nantinya algoritma *local search* perlu konsiderasikan ketika memilih paket yang ada. Selain itu, truk sendiri juga mempunyai dimensi peti kemas dan kapasitas maksimum yang dapat dibawa. Pengaturan bagaimana cara paket ditempatkan ke dalam peti kemas tentunya juga memiliki beberapa aturan, yakni paket di dalam peti kemas tidak boleh saling bertumpang tindih, dimensi paket tidak boleh keluar dari batas dimensi truk, paket yang tidak berada di lantai harus memperoleh dukungan dari paket di bawahnya, paket yang rapuh tidak boleh menjadi penopang, serta berat paket di dalam peti kemas tidak boleh melampaui kapasitas truk.

Penyelesaian untuk permasalahan *bin packing problem* tersebut nantinya akan menggunakan varian algoritma *local search* berupa Hill-climbing, Simulated Annealing, dan Genetic Algorithm yang nantinya hasil dari kualitas solusi dan waktu pencarian akan dihasilkan. Selain itu, program juga dapat menyajikan visualisasi plot keadaan awal, keadaan akhir, dan perubahan nilai objektif selama iterasi serta hasil akhir penempatan paket di dalam truk tersebut.

Pembahasan

A. State & Objective function

State pada program digunakan untuk menggambarkan kondisi semua paket di dalam dan di luar truk. Pada implementasi sendiri digunakan suatu dict key-value di mana Setiap ID paket direpresentasikan sebagai key dalam string, sedangkan value berisikan data model Placement, di mana akan merepresentasikan titik lokasi dalam x, y, dan z beserta orientasinya. ID paket sendiri sudah menyimpan data-data paket yang dibutuhkan via model Package yang sudah menyimpan data Package(id), sebagai representasi ID paket. Setelah itu, ada juga model Truck untuk merepresentasikan kapasitas dan dimensi truk yang nanti menjadi tempat paket dimasukkan. Model paket, truk, dan state dibuat dengan *dataclass* seperti pada potongan kode berikut.

```python
@dataclass(frozen=True)
class Package:
    id: str
    width: int
    length: int
    height: int
    value: int
    weight: int
    fragile: bool
    eta: int

@dataclass(frozen=True)
class Truck:
    width: int
    length: int
    height: int
    max_capacity: int

class Orientation(Enum):
    WLH = ("w", "l", "h")
    WHL = ("w", "h", "l")
    LWH = ("l", "w", "h")
    LHW = ("l", "h", "w")
    HWL = ("h", "w", "l")
```

```
    HLW = ("h", "l", "w")

@dataclass(frozen=True)
class Placement:
    x: int
    y: int
    z: int
    orientation: Orientation

@dataclass(frozen=True)
class State:
    placements: dict[str, Placement | None]
```

Agar algoritma *local search* nanti dapat menilai suatu state dengan state lainnya, diperlukan suatu Objective Function yang nantinya menilai bedasarkan penempatan dan penaltinya. Implementasi sendiri awalnya mengambil total nilai paket yang ditempatkan di dalam truk. Setelah itu, program mengurangi nilai tersebut dengan penalti untuk setiap pelanggaran penempatan atau kapasitas yang dimiliki. Secara gambar besar, implementasi digunakan menggunakan cara berikut

```
total_value = 0.0
total_weight = 0
placed_value = 0.0

for package_id, placement in state.placements.items():
    if placement is None:
        continue

    package = packages[package_id]
    size = dimensions(package, placement.orientation)
    package_volume = volume(package)
    value_per_cube = package.value / package_volume if package_volume else 0
    total_value += package.value
    placed_value += package.value
    total_weight += package.weight

    outofbounds_value = outOfBounds(placement, size, truck)
    overlap_value = 0
```

```
    support_value = 0
    fragile_value = 0

    for other_id, other_placement in state.placements.items():
        if other_placement is None or other_id == package_id:
            continue

        other = packages[other_id]
        other_size = dimensions(other, other_placement.orientation)
        overlap_value += overlaps(placement, size, other_placement, other_size)

        if placement.z != 0:
            support_value += supports(other_placement, other_size, placement, size)
            if other.fragile:
                fragile_value += supports(other_placement, other_size, placement, size)

    total_value -= outofbounds_value * value_per_cube * 1.1
    total_value -= overlap_value * value_per_cube * 1.1

    if placement.z != 0 and support_value <= 0:
        total_value -= package.value * 1.3

    if fragile_value >= 0:
        total_value -= fragile_value * size[2] * package.value * 1.5

if total_weight > truck.max_capacity:
    excess_weight = total_weight - truck.max_capacity
    total_value -= placed_value * (excess_weight / truck.max_capacity)
```

B. Pembangkitan Neighbor

Neighbor adalah state baru yang dibuat dengan melakukan perubahan posisi atau orientasi paket dari state saat ini. Perubahan yang dapat dilakukan adalah menukar posisi dua paket, memindahkan paket, mengeluarkannya dari truk, memasukkan paket yang berada di luar, atau memutar paket 90°. Selain itu, perubahan state dapat dilakukan dengan cara mencari posisi baru untuk suatu state paket dengan cara program mencoba koordinat bilangan bulat pada lebar dan panjang peti kemas pada paket yang dapat diletakkan di lantai truk atau di atas paket lain yang bisa menopangnya. Setelah itu, setiap kandidat diperiksa agar tetap valid dan tidak sama dengan neighbor yang sudah dibuat. Pemeriksaan ini dilakukan oleh fungsi add_neighbor() dengan implmentasi sebagai berikut.

```
def add_neighbor(truck: Truck, packages: dict[str, Package], candidate:
State, package_ids: list[str], seen: set[tuple[Placement | None, ...]],
neighbors: list[State]) -> None:
    key = tuple(candidate.placements[pid] for pid in package_ids)
    if key not in seen and is_valid_placement(truck, packages, candidate):
        seen.add(key)
        neighbors.append(candidate)
```

Masing-masing algoritma *local search* sendiri memiliki cara masing-masing dalam membangkitkan *negihbor*-nya, contohnya pada Hill-climbing membangkitkan semua neighbor yang valid, kemudian membandingkan nilainya dengan state saat ini. Untuk Simulated Annealing, pembangkitan *neighbor* menggunakan cara memilih satu perubahan secara acak di setiap iterasi melalui fungsi generate_neighbor(). Jika paket yang dipilih berada di luar truk, program mencoba memasukkannya pada posisi dan orientasi acak. Jika paket sudah berada di dalam, program dapat mengeluarkan atau memindahkannya dengan catatan *neighbor* yang tidak berubah atau tidak valid akan dilewati.

C. Implementasi Algoritma

a. Hill-Climbing

Hill Climbing merupakan algoritma *local search* yang selalu bergerak ke *neighbor* yang memiliki *score* yang lebih baik daripada *current state*. *Neighbor* merupakan *state* lain yang bisa dicapai dari *current state* dengan satu langkah yang dilakukan. Pada implementasi yang telah kami buat, *neighbor* dibangkitkan dengan tiga jenis move, yaitu menukar dua posisi paket (*swap*), memindahkan satu paket ke koordinat lain atau mengeluarkannya dari truk (*move*), dan merotasi paket sebesar 90 derajat terhadap sumbu x, y, atau z (*rotate*). Setiap *neighbor* yang telah dibangkitkan dan valid akan dicek dengan is_valid_placement untuk dievaluasi. Selain itu, kami juga menggunakan satu heuristik yang bertujuan untuk mengurangi jumlah *neighbor*, yaitu koordinat z yang dicoba hanya lantai truk (z = 0) dan permukaan atas dari paket lain yang tidak *fragile*, sehingga paket tidak akan dicoba saat pada posisi melayang.

I. Steepest Ascent Hill-Climbing

Pada setiap iterasi, Steepest Ascent akan mengevaluasi seluruh *neighbor* yang valid dari *current state*, lalu memilih *neighbor* dengan *score* yang paling tinggi. Jika *score* dari *neighbor* tersebut lebih tinggi daripada *score* dari *current state*, maka *current state* yang sekarang akan berpindah ke *neighbor* tersebut. Apabila tidak ada lagi *neighbor* yang *score*-nya lebih baik daripada *current state*, maka proses pencarian akan berhenti karena sudah mencapai *local optimum*.

Berikut merupakan gambaran kasar implementasi kode Steepest Ascent Hill-Climbing

```
while max_iterations is None or iteration < max_iterations:
    best_neighbor = None
    best_neighbor_value = float("-inf")
          for neighbor in generate_neighbors(truck, packages, current_state):
            neighbor_value = objective_function(truck, packages, neighbor)
```

```
        if neighbor_value > best_neighbor_value:
            best_neighbor = neighbor
            best_neighbor_value = neighbor_value

    iteration += 1
            if best_neighbor is not None and
best_neighbor_value > current_value:
        current_state = best_neighbor
        current_value = best_neighbor_value
    else:
        stop_reason = "local_optimum"
        break
```

II. Hill-Climbing with Sideways Move

Hill-Climbing with Sideways Move sebenarnya mirip dengan Steepest Ascent yang biasa, bedanya adalah *neighbor* terbaik yang *score*-nya sama dengan *current state*, masih tetap dapat diterima (*sideways move*). Tujuan dari *sideways move* adalah supaya proses *searching* bisa melewati *plateau* atau *shoulder*, yaitu daerah yang memiliki banyak *state* dengan *score* yang sama yang membuat Steepest Ascent langsung berhenti. Supaya proses *searching* tidak *looping* secara terus menerus di *plateau*, kami menambahkan parameter max_sideways_moves, agar *searching* bisa langsung berhenti jika jumlah dari *sideways move* sudah mencapai batas tersebut atau jika *neighbor* terbaik lebih buruk daripada *current state*.

Berikut merupakan gambaran kasar implementasi kode Hill-Climbing with Sideways Move.

```
if best_neighbor is None or best_neighbor_value <
current_value:
    stop_reason = "local_optimum"
elif best_neighbor_value == current_value and
sideways_moves >= max_sideways_moves:
    stop_reason = "max_sideways_moves"
else:
```

```
            is_sideways = best_neighbor_value ==
current_value
    current_state = best_neighbor
    current_value = best_neighbor_value
    if is_sideways:
        sideways_moves += 1
```

III. Random-Restart Hill-Climbing

Bedanya dengan Hill-Climbing with Sideways Move, Random-Restart Hill Climbing menjalankan Steepest Ascent beberapa kali, yaitu pada saat setiap kali Steepest Ascent berhenti di *local optimum*, proses *searching* diulang dari *initial state* random yang baru. Run pertama menggunakan *initial state* yang diberikan, sedangkan run berikutnya menggunakan *state* random dari generateRandomState. Jumlah restart-nya juga dibatasi oleh parameter max_restarts. Sehingga dari semua run yang telah dilakukan, *state* yang memiliki *score* tertinggi akan dipilih sebagai *final state*.

Berikut merupakan gambaran kasar implementasi kode Random-Restart Hill-Climbing.

```
for restart in range(max_restarts + 1):
    run_initial_state = initial_state if restart ==
0 else None
        result = steepest_ascent(truck, packages,
run_initial_state,                  max_iterations,
objective_function)

iterations_per_restart.append(result["iterations"])

    if best_state is None or result["final_value"] >
best_value:
        best_state = result["final_state"]
        best_value = result["final_value"]
        best_restart = restart
```

IV. Stochastic Hill-Climbing

Stochastic Hill-Climbing tidak mengevaluasi semua *neighbor* yang dibangkitkan, tidak seperti tiga varian Hill-Climbing sebelumnya. Pada setiap iterasi, program hanya akan membangkitkan satu *neighbor* secara acak dengan memanggil fungsi generate_neighbor. Jika *neighbor* tersebut valid dan *score*-nya lebih tinggi daripada *current state*, maka *neighbor* tersebut akan menjadi *current state* yang baru. Jika tidak, maka *neighbor* tersebut akan dibuang dan program akan lanjut ke iterasi berikutnya. Karena tidak ada kondisi untuk *local optimum* yang bisa dideteksi secara langsung, maka *searching* akan berhenti setelah mencapai max_iterations.

Berikut merupakan gambaran kasar implementasi kode Stochastic Hill-Climbing.

```
for iteration in range(1, max_iterations + 1):
      neighbor = generate_neighbor(truck, packages,
current_state)
            if neighbor != current_state and
is_valid_placement(truck, packages, neighbor):
          neighbor_value = objective_function(truck,
packages, neighbor)
        if neighbor_value > current_value:
            current_state = neighbor
            current_value = neighbor_value
```

b. Simulated Annealing

Simulated Annealing merupakan algoritma *local search* yang akan mencoba menerima suatu state yang lebih buruk dengan suatu temperature dengan harapan dapat keluar dari local optimum yang dimiliki. Algoritma Simulated Annealing sendiri dimulai dari state awal, lalu memilih satu *neighbor* secara acak pada setiap iterasi. Jika neighbor valid, program membandingkan *score*-nya dengan *score* state saat ini. Neighbor dengan *score* lebih tinggi langsung diterima. Tetapi, neighbor dengan *score* lebih rendah masih bisa diterima dengan peluang

$$e^{\frac{-\Delta E}{T}}$$

Dengan catatan karena kita ingin memaksimalkan score, kita mengubah $-\Delta E$ tersebut menjadi

$$e^{\frac{\Delta score}{T}}$$

Dengan cara ini, kita dapat memberi kesempatan pada pencarian untuk mencoba susunan lain dan tidak langsung berhenti saat bertemu solusi lokal. Berikut gambaran kasar implementasi kode Simulated Annealing

```
neighbor = generate_neighbor(truck, packages, current_state)

delta = None
exp_delta_over_temp = None
probability = None
accepted = False

if neighbor.placements != current_state.placements:
    if is_valid_placement(truck, packages, neighbor):
        neighbor_value = score(truck, packages, neighbor)

        delta = neighbor_value - current_value
        exponent = delta / temperature
        exp_delta_over_temp = math.exp(min(exponent, 700.0))
        probability = min(1.0, exp_delta_over_temp)

        accepted = random.random() < probability
        if accepted:
            current_state = neighbor
            current_value = neighbor_value

if current_value > best_value:
    best_state = State(current_state.placements.copy())
    best_value = current_value

temperature *= cooling_rate
```

c. Genetic Algorithm

Algoritma ini menyimulasikan penurunan genetik hasil perkawinan antara dua individu pada *state* yang ada pada populasi. Dalam algoritma ini, setiap iterasi pencarian memiliki populasi awal yang akan "dikawinkan", yaitu mencampur ciri-ciri satu *state* dengan *state* lain sehingga menghasilkan keturunan (*offspring*). Pelaksanaan algoritma ini secara umum memiliki 4 fase: membangkitkan populasi, memilih induk yang akan dikawinkan berdasarkan skor *fitness*, mengawinkan (*cross-over*), dan memutasi keturunan.

i. Fase pembangkitan

Populasi dibangkitkan secara *random* dengan jumlah individu di dalamnya sesuai dengan konfigurasi yang dimasukkan. Pada implementasi ini, populasi yang dibangkitkan akan selalu memiliki *state* yang valid (tidak ada paket yang tumpang tindih pada ruang yang sama, tidak melayang, dan tidak di-*support* oleh paket yang *fragile*). Keputusan tersebut dipilih agar algoritma menghasilkan *state-state* yang valid atau mendekati valid pada iterasi-iterasi berikutnya sehingga mempercepat pencarian ke *state* terbaik.

```
...
    # Generate initial population
    if initial_state is None:
        population = [generateRandomState(truck,
packages) for _ in range(population_count)]
    else:
        if not is_valid_placement(truck, packages,
initial_state):
            raise ValueError("initial_state harus
memenuhi semua batasan penempatan yang telah ditetapkan")
        population = [initial_state] +
[generateRandomState(truck, packages) for _ in
range(population_count - 1)]
```

ii. Fase pemilihan

Setiap individu pada populasi memiliki kemungkinan yang berbeda untuk dipilih sebagai induk yang akan saling dikawinkan. Penentuan

kemungkinan ini menggunakan skor *fitness*, yaitu skor yang menentukan seberapa sesuai suatu *state* terhadap objektif. Perhitungan skor tersebut pada tiap *state* dilakukan dengan menghitung skor *state* berdasarkan fungsi objektif yang digunakan lalu dibagi dengan skor total seluruh *state*.

Ketika terdapat skor yang bernilai negatif, dilakukan penormalisasian terhadap semua skor *state* dengan menambah setiap skor dengan skor terendah di antara semua *state* agar semua *state* memiliki kemungkinan dipilih (walaupun yang bernilai negatif akan memiliki kemungkinan yang sangat kecil) sehingga pergerakan algoritma dalam pencarian lebih leluasa (tidak terpaku pada *state* dengan skor terbaik di iterasi tertentu).

```
...
    # Start iterating
    for iteration in range(iteration_count):
        total_value = 0
        # Obtain values for each state
        for i in range(population_count):
            values[i] = objective_function(truck,
packages, population[i])
            total_value += values[i]
...
        # Normalize values
        normalized_values = []
        normalized_total_value = 0
        min_value = min(values)
        if min_value < 0:
            for i in range(population_count):
                normalized_values.append(values[i] -
min_value)
            normalized_total_value = total_value -
min_value * population_count

        # Find fitness value
        fitness = [0.0 for _ in range(population_count)]
        if normalized_total_value <= 0:
```

```
            fitness = [1/population_count for _ in
range(population_count)]
        else:
            for i in range(population_count):
                fitness[i] = normalized_values[i] /
normalized_total_value
```

Pada implementasi, akan dipilih individu sehingga total induk ada sebanyak 75% dari banyak individu dalam populasi. Keputusan ini dipilih agar algoritma dapat menerapkan elitism: menjaga beberapa individu dengan skor tinggi agar ada di populasi selanjutnya.

iii. Fase perkawinan

Setiap induk yang terpilih akan dikawinkan secara berpasangan sehingga menghasilkan dua keturunan. Proses perkawinan dilakukan dengan mencampurkan konfigurasi (posisi dan orientasi) paket antarinduk. Pada implementasi, hal tersebut dilakukan dengan memberikan kemungkinan 50% akan terjadi penukaran konfigurasi suatu paket antarinduk. Dengan begitu, ciri-ciri induk satu dengan lain menjadi tercampur di setiap keturunan yang dihasilkan.

Karena jumlah induk hanya 75% populasi, hasil perkawinan akan memberikan keturunan sebanyak 75% jumlah individu pada populasi. Sisa 25% tersebut diisi menggunakan elitism, yaitu mengisinya dengan 25% individu dalam populasi dengan skor tertinggi. Dengan begitu, individu dengan skor tertinggi akan selalu terjaga antariterasi sehingga algoritma akan selalu bergerak menuju solusi optimal sesuai objektifnya. Berikut kode implementasi fase pemilihan dan perkawinan.

```
        # Cross-over
        crossover: list[State] = []
        offspring_count = population_count * 3 // 4 #
Will generate only 75% of population
        crossover_count = offspring_count // 2
        for _ in range(crossover_count):
```

```
            offsprings: list[State] =
[State(placements={}), State(placements={})]

            # Pick 2 parents from population with weight
from fitness value
            parents = random.choices(population, fitness,
k=2)

            # Randomize crossover, 50/50 chance to pick
either for each package placement
            for package_id, placement in
parents[0].placements.items():
                rand = random.random()
                if rand <= 0.5:
                    offsprings[0].placements[package_id]
= placement
                    offsprings[1].placements[package_id]
= parents[1].placements[package_id]
                else:
                    offsprings[0].placements[package_id]
= parents[1].placements[package_id]
                    offsprings[1].placements[package_id]
= placement

            crossover += offsprings

        # Populate crossover with highest valued states
(25% elitism)
        elite_count = population_count - len(crossover)
        ranked_indices = sorted(
            range(population_count),
            key=lambda index: values[index],
            reverse=True,
        )
        for index in ranked_indices[:elite_count]:
            crossover.append(population[index])
```

iv. Fase pemutasian

Mutasi pada suatu keturunan artinya mengubah sebagian keadaan keturunan secara acak tanpa pengaruh dari genetik induknya. Hal ini dilakukan agar ruang pencarian algoritma menjadi lebih luas karena

memungkinkan perubahan yang tidak tergantung pada populasi pada iterasi tertentu.

Pada implementasi, mutasi dilakukan pada setiap keturunan yang tidak valid sehingga ada kemungkinan keturunan tersebut menjadi valid, yaitu dengan mengganti keturunan dengan tetangganya (*state* yang konfigurasinya mendekati, dicapai dengan menukar konfigurasi 2 paket, memindahkan posisi paket, dan merotasi paket). Hasil perkawinan setelah mutasi akan menjadi populasi awal pada iterasi selanjutnya.

```
        # Mutation (50/50 happens to each state that is
invalid, will replace state to its random neighbor)
        for i in range(population_count):
            if not is_valid_placement(truck, packages,
crossover[i]) and random.random() < 0.5:
                crossover[i] = generate_neighbor(truck,
packages, crossover[i])

        population = crossover
```

D. Hasil Eksperimen

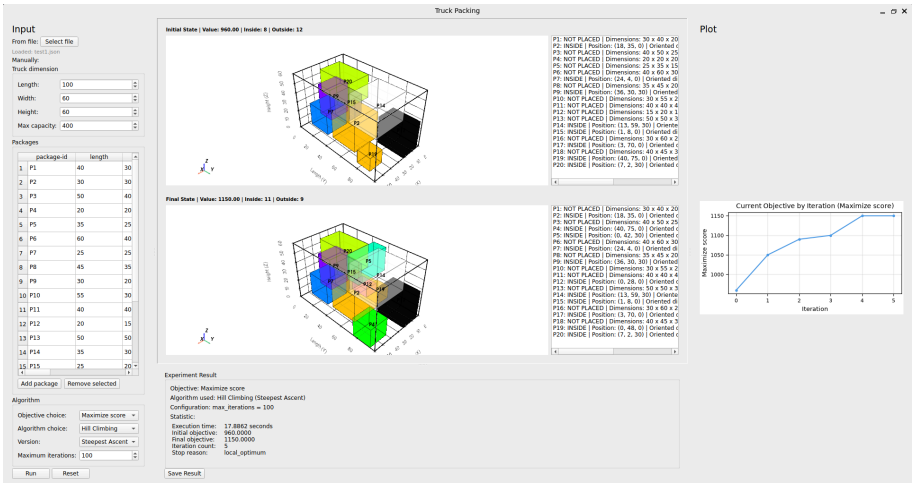
Eksperimen dilakukan pada satu konfigurasi input (truk dan paket-paket) dengan berbagai algoritma yang telah diimplementasikan. Eksperimen pada setiap algoritma dijalankan sebanyak 3 kali. Berikut adalah konfigurasi input yang dipakai.

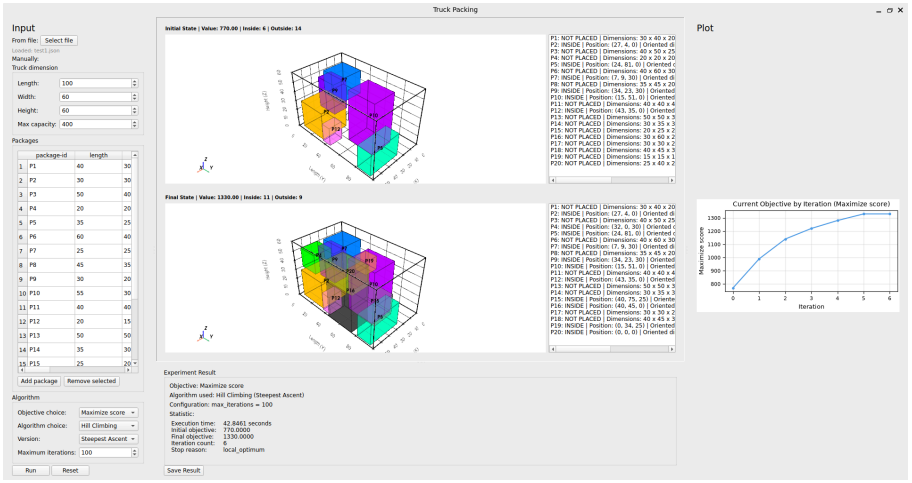

```
{
  "truck": { "length": 100, "width": 60, "height": 60, "max_capacity": 400 },
  "packages": [
    { "id": "P1",  "length": 40, "width": 30, "height": 20, "weight": 25, "value": 120, "fragile": false },
    { "id": "P2",  "length": 30, "width": 30, "height": 30, "weight": 40, "value": 200, "fragile": false },
    { "id": "P3",  "length": 50, "width": 40, "height": 25, "weight": 55, "value": 300, "fragile": false },
    { "id": "P4",  "length": 20, "width": 20, "height": 20, "weight": 10, "value": 60,  "fragile": true },
    { "id": "P5",  "length": 35, "width": 25, "height": 15, "weight": 18, "value": 90,  "fragile": false },
    { "id": "P6",  "length": 60, "width": 40, "height": 30, "weight": 70, "value": 450, "fragile": false },
    { "id": "P7",  "length": 25, "width": 25, "height": 25, "weight": 22, "value": 110, "fragile": true },
    { "id": "P8",  "length": 45, "width": 35, "height": 20, "weight": 38, "value": 180, "fragile": false },
    { "id": "P9",  "length": 30, "width": 20, "height": 15, "weight": 12, "value": 70,  "fragile": true },
    { "id": "P10", "length": 55, "width": 30, "height": 25, "weight": 48, "value": 260, "fragile": false },
    { "id": "P11", "length": 40, "width": 40, "height": 40, "weight": 60, "value": 350, "fragile": false },
    { "id": "P12", "length": 20, "width": 15, "height": 10, "weight": 5,  "value": 40,  "fragile": true },
    { "id": "P13", "length": 50, "width": 50, "height": 30, "weight": 65, "value": 400, "fragile": false },
    { "id": "P14", "length": 35, "width": 30, "height": 30, "weight": 35, "value": 170, "fragile": false },
    { "id": "P15", "length": 25, "width": 20, "height": 20, "weight": 15, "value": 80,  "fragile": true },
    { "id": "P16", "length": 60, "width": 30, "height": 20, "weight": 45, "value": 220, "fragile": false },
    { "id": "P17", "length": 30, "width": 30, "height": 20, "weight": 28, "value": 130, "fragile": false },
    { "id": "P18", "length": 45, "width": 40, "height": 35, "weight": 58, "value": 310, "fragile": false },
    { "id": "P19", "length": 15, "width": 15, "height": 15, "weight": 6,  "value": 50,  "fragile": true },
    { "id": "P20", "length": 40, "width": 25, "height": 25, "weight": 32, "value": 150, "fragile": false }
  ]
}
```

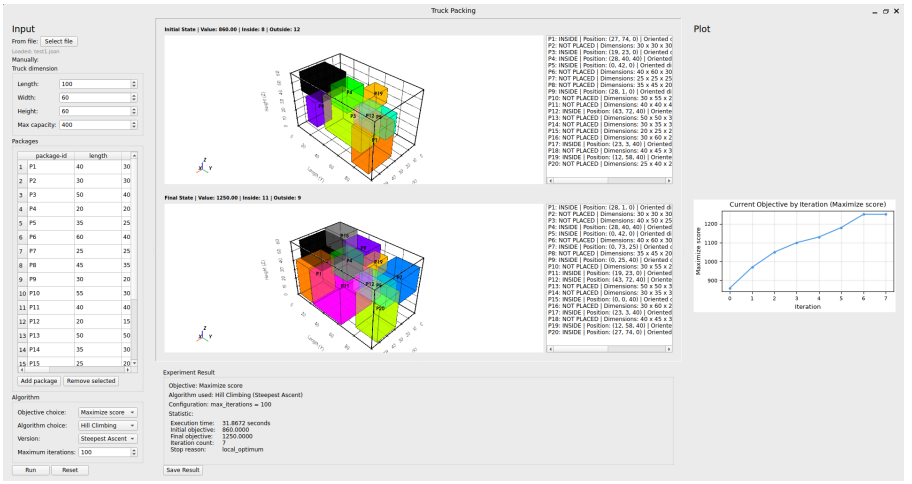

a. Hill Climbing

i. Steepest Ascent Hill-Climbing

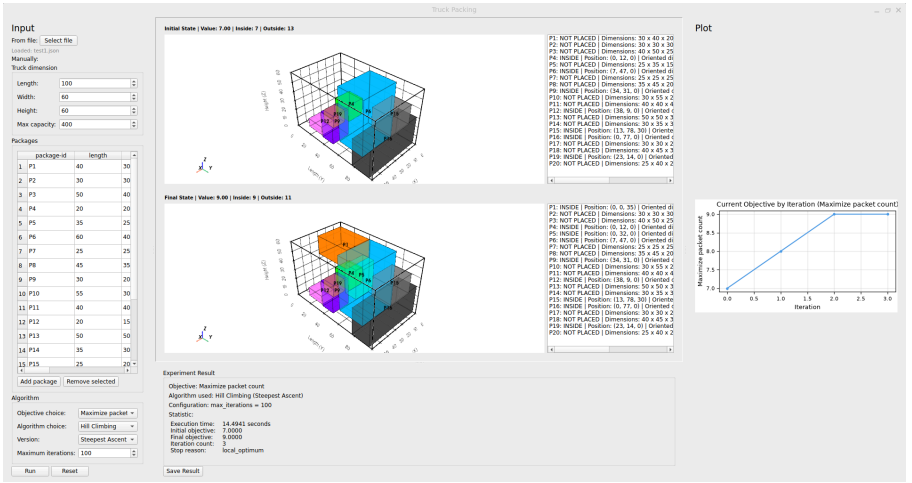
Objektif: Maximum score

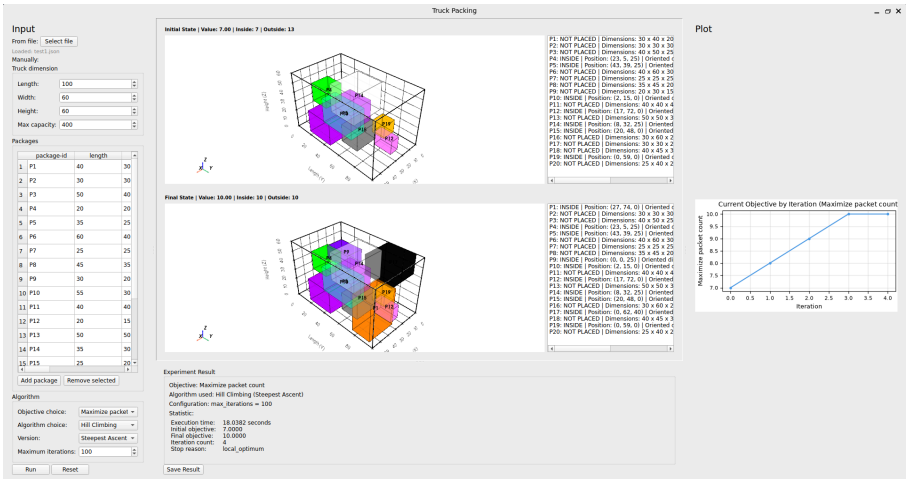


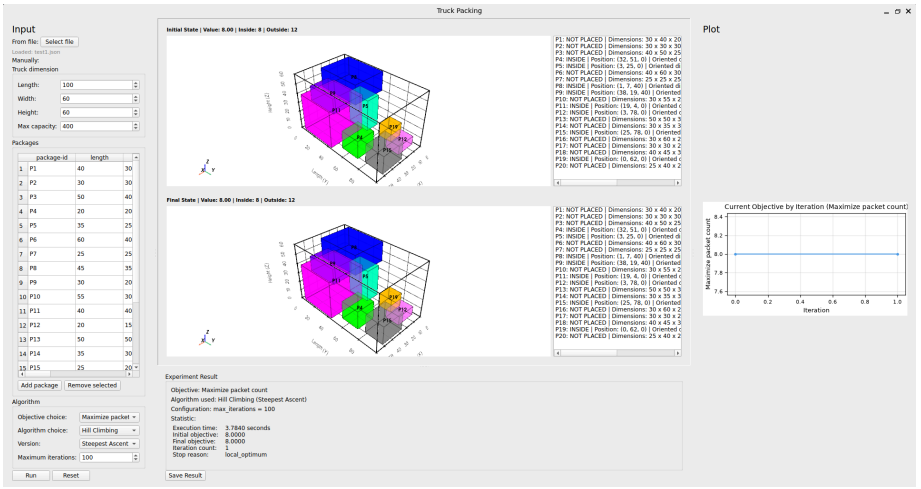




Objektif: Maximum packet count

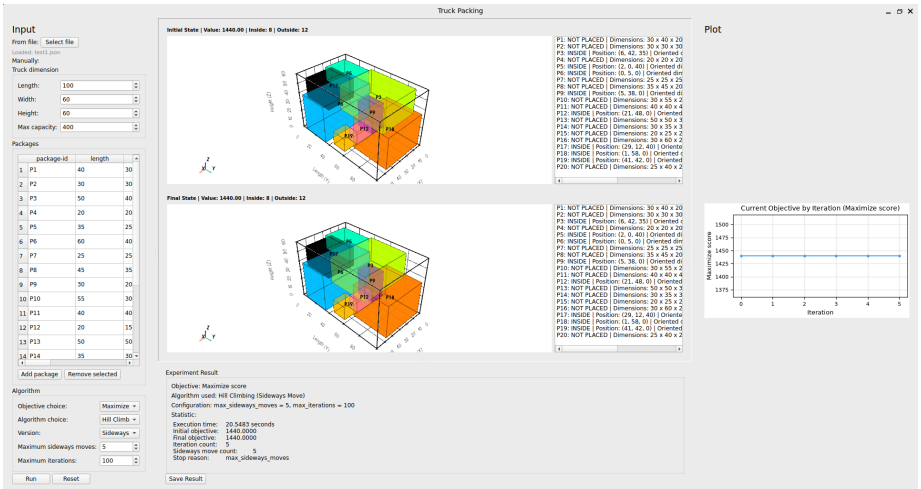


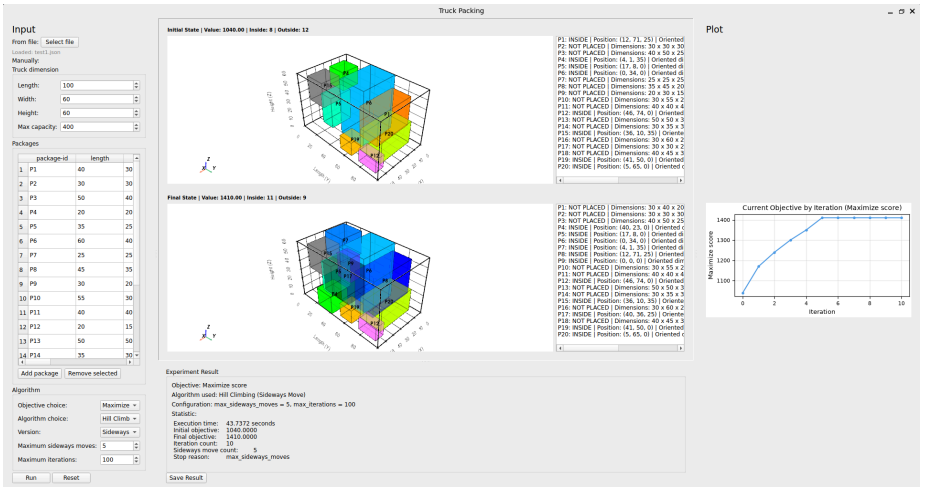


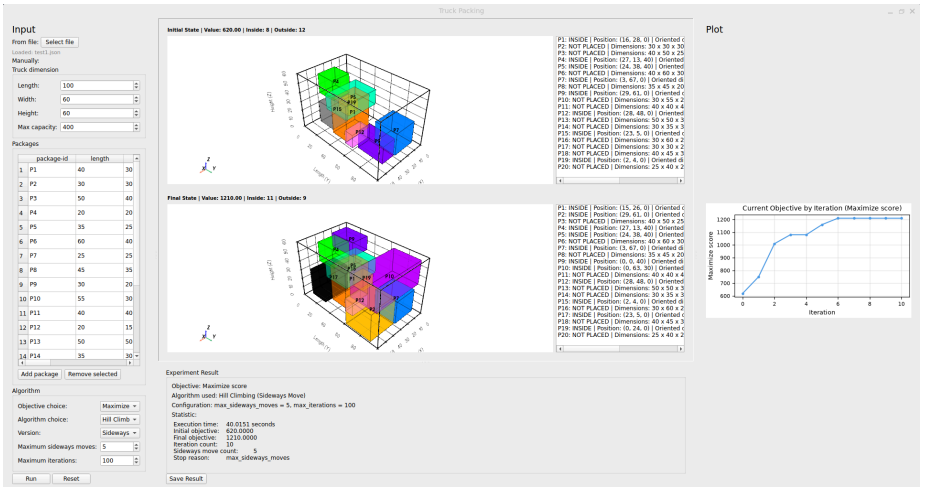


ii. Hill-Climbing with Sideways Move

Objektif: Maximum score

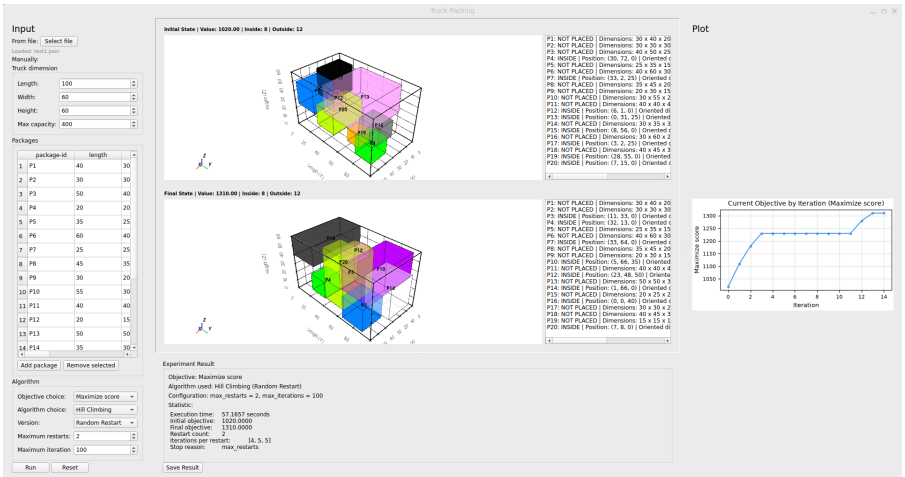


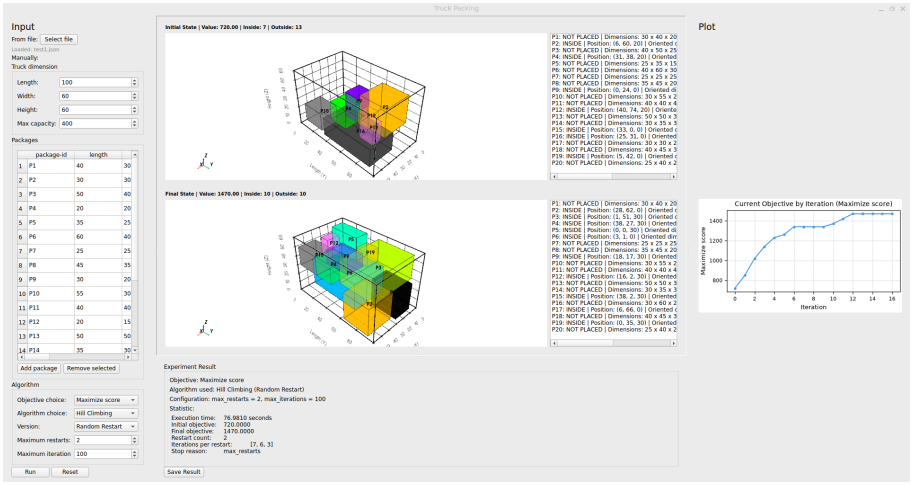


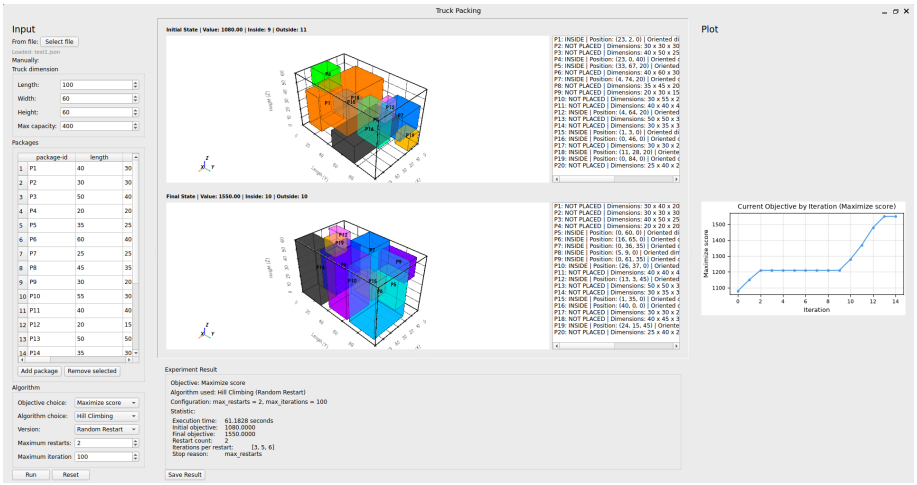


iii. Random-Restart Hill-Climbing

Objektif: Maximum score

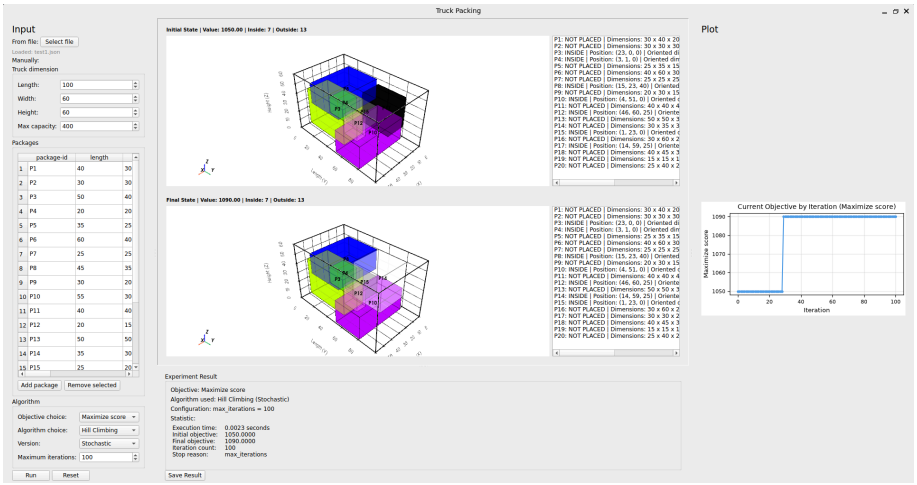


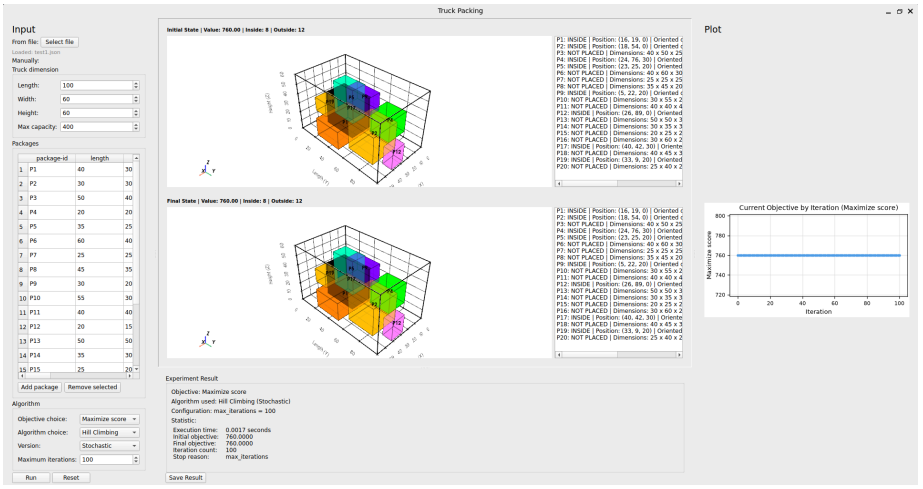


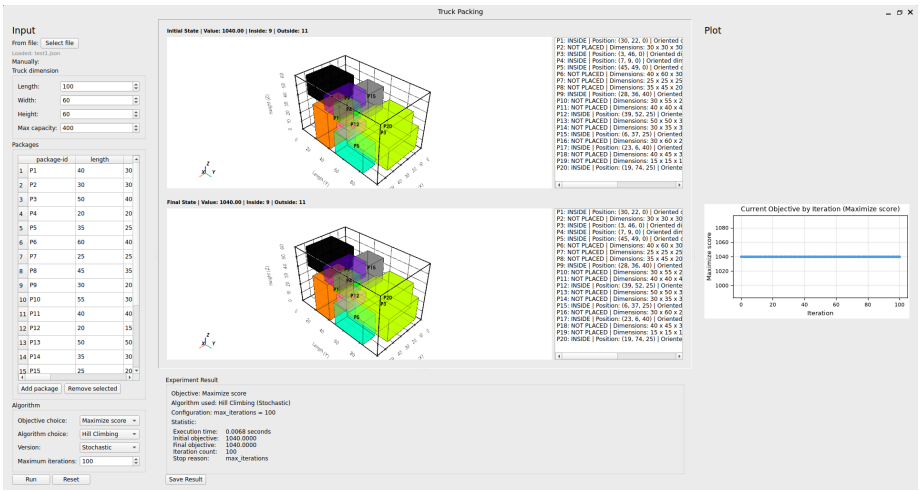


iv. Stochastic Hill-Climbing

Objektif: Maximum score

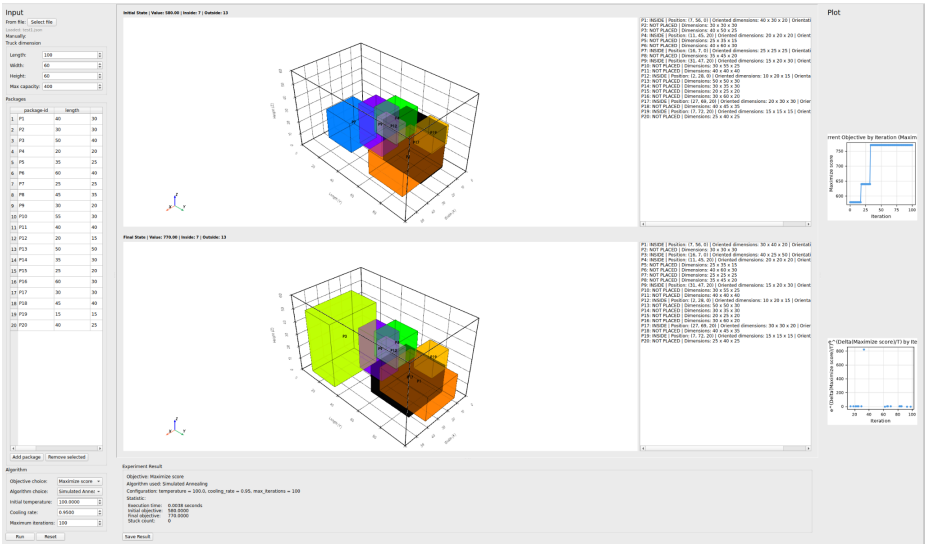


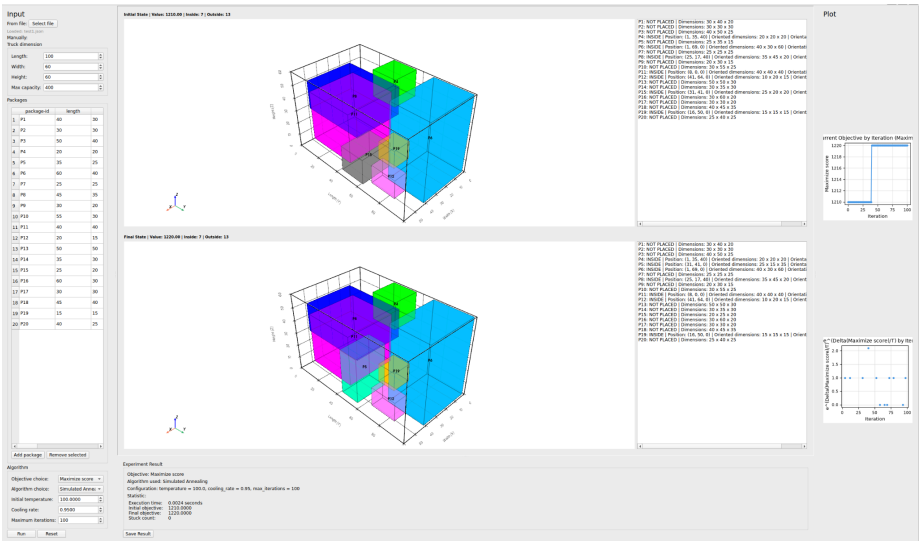


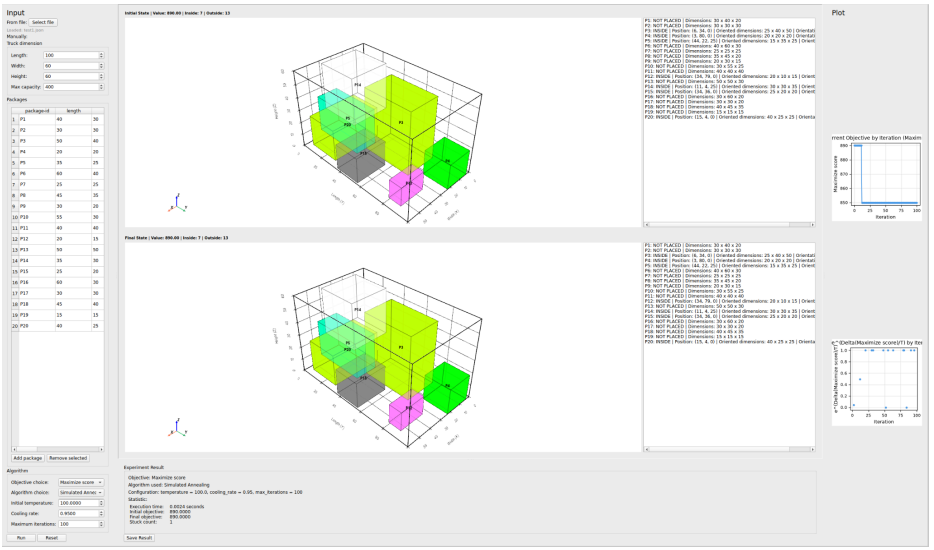


b. Simulated Annealing

Objektif: Maximum score

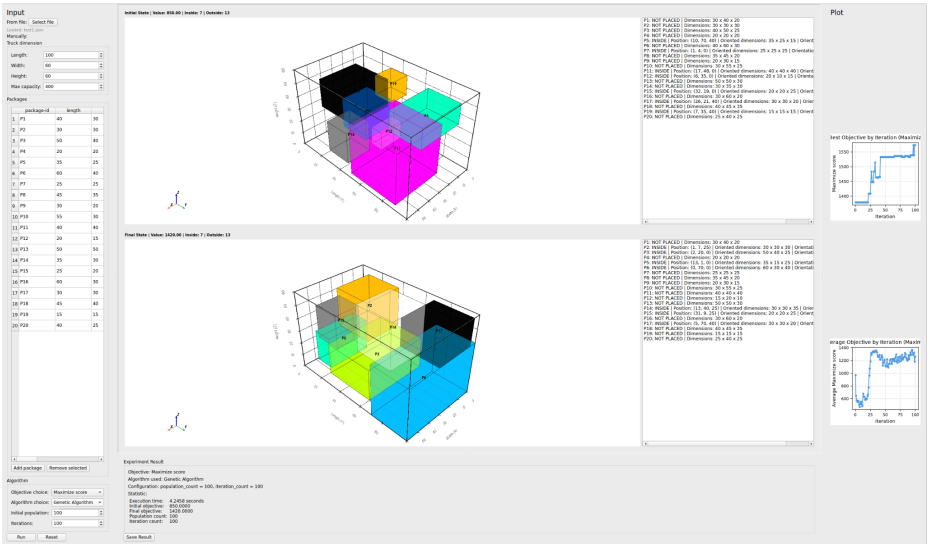


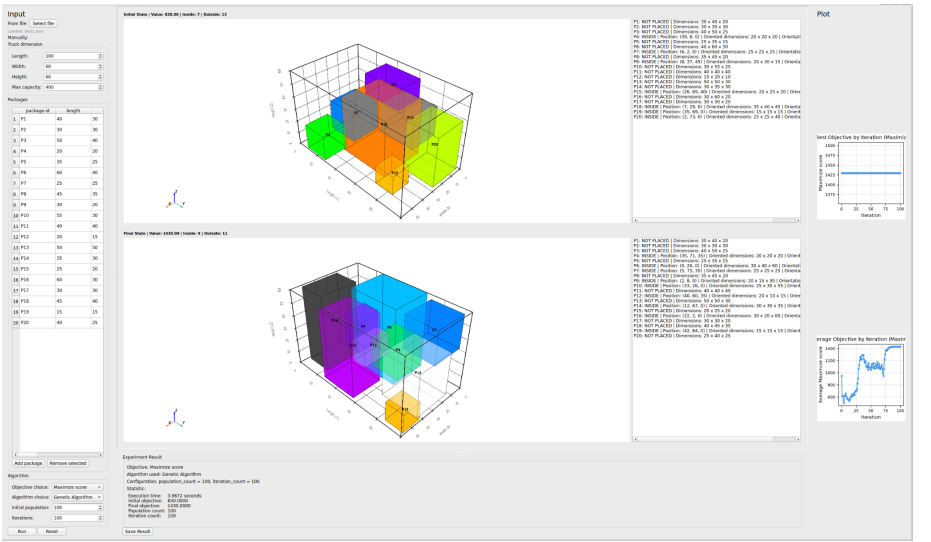


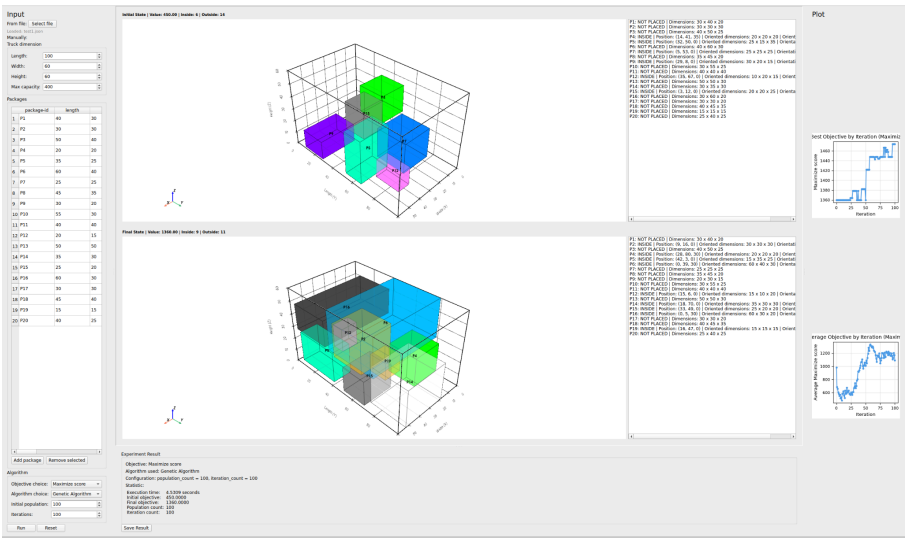


c. Genetic Algorithm

Objektif: Maximum score


Input
Plot
Packages
Algorithm
Experiment Result


Input
Plot
Packages
Algorithm
Experiment Result


Input
Plot
Packages
Algorithm
Experiment Result

E. Analisis Eksperimen

Secara umum, algoritma Hill-Climbing memiliki waktu eksekusi yang sangat besar, kecuali pada versi Stochastic. Hal ini karena versi Hill-Climbing selain Stochastic dalam satu iterasi melakukan banyak generasi *neighbor state* yang sangat banyak, sehingga per iterasi butuh waktu eksekusi yang lama. Di sisi lain, Hill-Climbing versi Stochastic sangat cepat dilakukan karena setiap iterasi hanya membangkitkan satu saja *neighbor state*. Namun, optimalnya Stochastic harus memiliki iterasi yang sangat besar untuk mendapatkan *state* baru dengan skor yang lebih dari *initial state*. Algoritma Hill-Climbing kebanyakan terjadi *stuck* pada *local optima*, yang bisa dilihat dari tidak adanya perubahan skor dari *initial state*

Algoritma Simulated Annealing juga mirip dengan Hill-Climbing Stochastic, eksekusi cepat karena setiap iterasi perlu sedikit. Berbeda dengan Hill-Climbing Stochastic, *state* yang lebih buruk bisa saja dipilih menjadi *state successor*.

Algoritma Genetic Algorithm memiliki eksekusi cukup cepat namun tidak secepat Stochastic ataupun Simulated Annealing. Terlihat bahwa rata-rata skor naik seiring iterasi bertambah. Hal ini karena diterapkannya elitism pada Genetic Algorithm sehingga *state* dengan skor tinggi tidak tergantikan oleh keturunan.

F. Implementasi Bonus

a. Varian Alternatif Objective Function

Pada spesifikasi, objective function utama digunakan untuk memaksimalkan total nilai paket yang dimuat ke dalam truk yang direpresentasikan sebagai score(). Sebagai objective function alternatif, program juga menyediakan pilihan untuk memaksimalkan jumlah paket yang berhasil dimuat dengan membuat fungsi package_count(). Perhitungan jumlah paket dilakukan oleh fungsi yang terdapat pada snippet kode berikut.

```
def package_count(truck: Truck, packages: dict[str,
Package], state: State) -> int:
    count = 0
    for placement in state.placements.values():
        if placement is not None:
            count += 1

    return count
```

Fungsi ini melihat isi placements pada suatu state dimana setiap paket yang memiliki placement dihitung satu, sedangkan paket dengan nilai None tidak dihitung. Objective function ini tidak menambah variabel baru karena hanya menggunakan data penempatan paket yang sudah ada pada state.

Setelah itu, dibuat suatu parameter input objective_function yang meminta user melakukan input variasi objective function apa yang digunakan, “Maximize score” untuk tipe variasi objective function pada implementasi wajib, “Maximize package count” pada variasi alternatif ojective function bonus yang dibuat

```
objective_label = self.objective_choice.currentText()
   if objective_label == "Maximize score":
      objective_function = score
   elif objective_label == "Maximize packet count":
      objective_function = package_count
```

b. Implementasi Seluruh Algoritma Hill-climbing

Selain satu algoritma Hill-climbing yang diwajibkan, kami juga mengimplementasikan seluruh algoritma Hill-climbing, yaitu Steepest Ascent Hill-Climbing, Hill-Climbing with Sideways Move, Random-Restart Hill-Climbing, dan Stochastic Hill-Climbing. Seluruh algoritmanya menggunakan representasi *state*, fungsi pembangkit *neighbor*, dan *objective function* yang sama, sehingga hasil dari keempatnya bisa dibandingkan secara langsung. Penjelasan cara kerja dan *snippet code* dari setiap algoritma telah dibahas pada bagian Implementasi Algoritma Hill-Climbing.

Kesimpulan

Berdasarkan eksperimen yang telah dijalankan, (hasil eksperimen) algoritma dengan nilai objektif akhir tertinggi pada konfigurasi yang diuji adalah (isi) sedangkan waktu pencarian terendah diperoleh (isi). Hasil tersebut berlaku untuk data dan parameter eksperimen pada laporan ini dan tidak dengan sendirinya membuktikan solusi global.

Dari pengerjaan tugas besar ini, disarankan untuk mengotak-atik lebih lanjut konfigurasi terbaik untuk implementasi tiap algoritma, terutama pada algoritma Simulated Annealing dan Genetic Algorithm.

Referensi

[1] “How to Implement Randomization with the Python Random Module,” *Towards Data Science*. [Online]. Available: https://towardsdatascience.com/implementing-randomisation-with-the-python-random-module/.

[2] “Simulated Annealing,” *Algorithms for Competitive Programming*. Last updated May 24, 2026. [Online]. Available: https://cp-algorithms.com/num_methods/simulated_annealing.html

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 19, 2026].

Lampiran

A. Tabel Pembagian Tugas

Nama	NIM	Tugas yang dikerjakan
Faiq Azzam Nafidz	13524003	Objective function utama (score()), Genetic Algorithm, GUI, visualisasi *state*, plotting, dan laporan.
Anindya Naufal Pinasthika	13524013	State & Objective function, pembangkitan neighbor, simulated annealing, bonus alternatif objective function, dan laporan
Azri Arzaq Pohan	13524139	State, hill climbing (Steepest Ascent), bonus implementasi seluruh algoritma hill-climbing (Sideways Move, Random-Restart, Stochastic), dan laporan

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Faiq Azzam Nafidz/13524003, Anindya Naufal Pinasthika/13524013, Azri Arzaq Pohan/13524139
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Mengecek konsistensi penulisan dan kejelasan kalimat dalam pembuatan laporan
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Diskusi desain implementasi terbaik, rekomendasi *library* visualisasi, pembangkitan *test case*, membantu *debugging* program
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:	-	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 17 Oktober 2026





Faiq Azzam Nafidz 13524003 Anindya Naufal Pinasthika 13524013 Azri Arzaq Pohan 13524139